NSBM GREEN UNIVERSITY

Faculty of Computing

Bachelor of Science in Software Engineering

INTERIM SUBMISSION – 01

AUTOMATED CLOUD INFRASTRUCTURE RECOMMENDATION SYSTEM WITH TERRAFORM AND DOCKER GENERATION FOR WEB APPLICATIONS

Submitted by:

Mahamalage Yasindu Binod Perera

Student ID: 28556

Final Year Research Project

Chapter 01 – Introduction 1
1.1 Chapter Overview 2
1.2 Problem Background 2
1.3 Problem Statement 2
1.3.1 General Problem 3
1.3.2 Specific Problem 3
1.4 Research Question 3
1.5 Research Motivation 4
1.6 Research Aim 4
1.7 Research Objectives 4
1.7.1 To Identify 4
1.7.2 To Analyse 4
1.7.3 To Design and Develop 5
1.7.4 To Evaluate 5
1.8 Rich Picture of the Proposed Solution 5
1.9 Resource Requirements 5
1.9.1 Hardware 5
1.9.2 Software 6
1.10 Project Scope 6
1.11 Chapter Summary 7
Chapter 02 – Literature Review 7
2.1 Chapter Overview 8
2.2 Conceptual Map of the Literature 8
2.3 Domain Overview 8
2.4 Existing Systems, Frameworks, and Designs 9
2.4.1 Sharma et al. – Machine Learning-Based Resource Recommendation [4] 9
2.4.2 Chen et al. – Static Code Analysis for Memory Estimation [5] 9
2.4.3 Zhang and Wang – CloudCostOpt [6] 9
2.4.4 Kim et al. – TerraGen [7] 10
2.4.5 Patel and Singh – Multi-Cloud Deployment Framework [9] 10
2.5 Technological Analysis 11
2.5.1 Algorithmic Analysis 11
2.5.2 Design Analysis 11
2.5.3 Workflow Analysis 11
2.6 Reflection – Research Gap Justification 12
Chapter 03 – Methodology 12
3.1 Chapter Overview 13
3.2 Research Paradigm 13
3.3 Research Approach 13

3.4 Research Strategy 14
3.5 Fact Collection Mechanisms 14
3.6 Research Methodology Execution Workflow 14
3.7 Project Management Methodology 15
3.8 Project Timeline 16
3.9 Ethical Considerations 16
3.10 Chapter Summary 17
References 17

Chapter 01 – Introduction

1.1 Chapter Overview

This chapter establishes the foundational context for the research project. It opens by presenting the background and the problem that motivated this study, followed by a clearly defined problem statement and research question. The chapter then articulates the research motivation, aim, and objectives that guide the direction of the project. A rich picture of the proposed solution is included to offer a visual overview of the system's intended workflow. The chapter also outlines resource requirements, defines the scope of the project, and closes with a brief summary.

1.2 Problem Background

Cloud computing has become the dominant paradigm for deploying modern software applications, with major providers such as Amazon Web Services (AWS), Microsoft Azure, Google Cloud Platform (GCP), and DigitalOcean collectively serving millions of organizations worldwide. This shift has radically changed the responsibilities of software developers, who are now expected not only to build applications but also to make informed decisions about the cloud infrastructure on which those applications will run.

The scale and complexity of modern cloud offerings present a significant barrier to developers. AWS alone offers over 400 distinct EC2 instance types across multiple performance families, each optimized for different computational profiles. Beyond compute, developers must also configure storage, networking, security groups, managed services, and cost structures — often simultaneously and under time pressure. What was once a specialized DevOps responsibility has increasingly fallen on development teams who lack the operational expertise to navigate these decisions efficiently.

This challenge is compounded by the maturation of Infrastructure as Code (IaC) tooling, particularly Terraform, and containerization technologies such as Docker. While these technologies bring reproducibility and scalability to infrastructure management, they introduce yet another layer of expertise that many developers have not acquired. The consequence is a pattern of either under-provisioned infrastructure that fails under production load, or over-provisioned resources that waste organizational budgets. Industry estimates suggest that approximately 45% of total cloud expenditure is wasted on idle or incorrectly sized resources, amounting to billions of dollars annually across the industry [2]. This situation reveals a critical, unaddressed need: an intelligent, automated tool that can translate application-level source code characteristics into precise, optimal cloud infrastructure recommendations — before the first line of infrastructure code is written.

1.3 Problem Statement

1.3.1 General Problem

Developers and organizations deploying web applications to the cloud routinely face the challenge of selecting appropriate infrastructure configurations without adequate automated guidance. The consequence of this gap is widespread — approximately 68% of developers report difficulty in selecting suitable cloud resources [1], and nearly half of all cloud spending is estimated to be wasted due to incorrect resource sizing [2]. At a high level, the absence of an integrated, pre-deployment recommendation system forces developers to rely on trial and error, manual research across thousands of pages of documentation, or informal expert consultation. This inefficiency drives up costs, increases time to deployment, and introduces the risk of performance failures in production environments. The problem is not isolated to individual projects; it represents a systemic inefficiency in the software industry's approach to cloud adoption.

1.3.2 Specific Problem

Within the domain of software engineering and cloud operations, the specific shortcoming is the absence of a system capable of analysing a software repository's source code and automatically generating accurate, multi-dimensional cloud infrastructure recommendations — prior to any deployment. Existing tools address only fragments of this problem: post-deployment cost optimizers [6] require running systems, static code analysers [5] address only single resource dimensions, and IaC template generators [7] rely on static patterns rather than application-specific intelligence. No existing solution integrates code analysis, machine learning-based resource prediction, multi-provider cost estimation, and automated IaC generation into a single cohesive pre-deployment workflow. This fragmentation leaves a clearly identifiable research gap — the need for an end-to-end, intelligent, pre-deployment cloud infrastructure recommendation system driven by source code analysis.

1.4 Research Question

How can an automated system effectively analyse software repository characteristics to generate optimal, secure, and cost-efficient cloud infrastructure recommendations with corresponding Infrastructure as Code configurations across multiple cloud providers?

Supporting sub-questions include:

- What repository-level features are most predictive of cloud resource requirements such as CPU, memory, and storage?
- How can machine learning models be effectively trained to map application characteristics to appropriate cloud instance types across AWS, Azure, GCP, and DigitalOcean?
- What techniques enable automatic generation of production-ready, security-compliant Terraform and Dockerfile configurations from code analysis outputs?

1.5 Research Motivation

The motivation for this research is grounded in direct professional experience. During an industry internship, the researcher was tasked with deploying a Django REST Framework API to an AWS EC2 instance. The process exposed four significant practical challenges: navigating over 400 instance types with no structured guidance; accurately estimating monthly costs before committing to a configuration; managing the risk of choosing insufficient resources for production workloads; and writing secure, optimized Terraform and Dockerfile configurations from scratch with limited prior exposure to IaC tooling.

This experience is not unique. The Cloud Native Computing Foundation's 2024 Annual Survey found that 72% of organizations consider cloud cost management a significant ongoing challenge, and 35% specifically identify resource right-sizing as a primary pain point [3]. From an academic perspective, the motivation lies in addressing a meaningful and underexplored intersection between software engineering and cloud infrastructure management — one where intelligent automation has the potential to substantially reduce developer cognitive load and organizational waste. There is also an environmental dimension: over-provisioned cloud resources consume unnecessary energy in data centres, and a more intelligent provisioning system could meaningfully contribute to sustainable computing practices.

1.6 Research Aim

This research aims to design, develop, and evaluate an automated cloud infrastructure recommendation system that analyses software repositories and produces optimal, provider-specific deployment configurations alongside fully generated Infrastructure as Code scripts — reducing developer effort, improving resource efficiency, and enabling cost-informed decision-making before any deployment occurs.

1.7 Research Objectives

1.7.1 To Identify

To identify the repository-level features — including programming languages, frameworks, dependency declarations, and architectural patterns — that are most predictive of cloud resource requirements such as compute, memory, storage, and networking.

1.7.2 To Analyse

To analyse existing approaches to cloud resource recommendation, IaC generation, and cost optimization in order to identify gaps, limitations, and opportunities for integration into a unified pre-deployment system.

1.7.3 To Design and Develop

To design and develop an end-to-end automated system comprising: a GitHub repository analyser, a machine learning-based recommendation engine, a multi-provider cost estimation module, a Terraform script generator, a Dockerfile generator, and a web-based user interface.

1.7.4 To Evaluate

To evaluate the accuracy and practical utility of the system's recommendations by comparing generated outputs against expert cloud infrastructure decisions, assessing generated code quality, and measuring system performance under realistic usage conditions.

1.8 Rich Picture of the Proposed Solution

The proposed system operates as an end-to-end pipeline triggered by a developer submitting a GitHub repository URL through the web interface. The repository analyser then clones or fetches the repository and extracts key signals: technology stacks detected from configuration files such as package.json, requirements.txt, and pom.xml; dependency graphs and framework versions; architectural patterns indicating monolithic, microservices, or serverless designs; and database and caching service references.

These extracted features are passed to the machine learning recommendation engine, which maps them to optimal cloud instance types across AWS, Azure, GCP, and DigitalOcean. Simultaneously, the cost estimation engine retrieves real-time pricing data from provider APIs and computes projected monthly costs including compute, storage, and supplementary services. The IaC generation modules then produce a Terraform configuration file and a Dockerfile, both incorporating security best practices and environment-specific configurations for development, staging, and production.

The web interface presents the developer with a structured comparison across providers — including cost breakdowns, recommended specifications, and generated code — all downloadable for immediate use. This workflow eliminates the need for manual research, reduces the time from code to deployable infrastructure, and ensures that recommendations are grounded in the specific characteristics of each application rather than generic defaults.

1.9 Resource Requirements

1.9.1 Hardware

Resource	Specification
Development Machine	Minimum 16 GB RAM, multi-core CPU (Intel i7 / AMD Ryzen 7 or equivalent)
GPU (for ML training)	NVIDIA GPU with CUDA support, or equivalent cloud-based GPU instance
Storage	Minimum 500 GB SSD for local repository cloning and dataset storage

Network	Stable internet connection for GitHub API, cloud pricing API, and deployment testing

1.9.2 Software

Software / Tool	Purpose / Version
Operating System	Ubuntu 22.04 LTS (development and deployment)
Backend Framework	Python 3.10+, FastAPI
Machine Learning	Scikit-learn, TensorFlow 2.x, XGBoost
Frontend	React.js 18, Material-UI
Database	PostgreSQL 15, MongoDB Atlas
IaC Tooling	Terraform CLI (for validation), Docker
CI/CD	GitHub Actions
Cloud APIs	AWS SDK (boto3), Azure SDK, GCP Client Libraries, DigitalOcean API
Version Control	Git, GitHub

1.10 Project Scope

The scope of this research is defined below. Items listed as in-scope form the deliverable boundaries of the project, while out-of-scope items are explicitly excluded to maintain focus and feasibility within the available project timeline.

In Scope	Out of Scope
Analysis of public GitHub repositories across common web frameworks (Django, Express, Spring, etc.)	Support for mobile, desktop, or embedded application types
Infrastructure recommendations for AWS, Azure, GCP, and DigitalOcean	Recommendations for other cloud providers (e.g., IBM Cloud, Oracle Cloud)
Generation of Terraform configurations and Dockerfiles	Generation of Kubernetes manifests or Helm charts
Pre-deployment cost estimation with real-time pricing	Ongoing post-deployment cost monitoring or alerting
Web-based user interface for repository input and result visualization	CLI tool or IDE plugin integration
Recommendation accuracy evaluation against cloud expert decisions	Large-scale production deployment of the system beyond evaluation scope
Support for multi-environment configurations (dev, staging, prod)	Automatic deployment execution to cloud environments

1.11 Chapter Summary

This chapter introduced the research by situating it within the broader context of cloud infrastructure complexity and the growing challenges developers face during deployment planning. The background highlighted the scale of the problem, from the abundance of cloud options to the widespread misallocation of cloud resources. The problem statement was defined at both a general and a discipline-specific level, culminating in the identification of the research gap that this project addresses. A focused research question was formulated, followed by motivation, aim, and four structured objectives. A rich picture illustrated the intended system workflow; resource requirements were outlined, and the project's scope was clearly bounded. The following chapter presents a critical review of the literature relevant to this research.

Chapter 02 – Literature Review

2.1 Chapter Overview

This chapter presents a structured and critical review of literature relevant to the automated cloud infrastructure recommendation domain. It begins with a conceptual map that illustrates how reviewed works relate to one another and to this research. A domain overview establishes foundational theoretical ground, followed by a comparative assessment of existing systems and frameworks. The technological analysis — the core of the chapter — examines algorithms, design patterns, and workflows employed in related work. The chapter concludes with a reflective synthesis that identifies and justifies the research gap addressed by this study.

2.2 Conceptual Map of the Literature

The literature reviewed in this chapter is organized across four interconnected themes, each contributing to a different component of the proposed system. The first theme, Cloud Resource Management, encompasses studies on resource recommendation, workload prediction, and cost optimization. The second, Code and Repository Analysis, covers static analysis techniques and dependency-based infrastructure estimation. The third theme, Infrastructure as Code, addresses IaC generation approaches, security in Terraform scripts, and multi-cloud orchestration. The fourth, Containerization and Deployment Optimization, includes work on Docker image optimization and performance benchmarking. These four themes converge at the research gap: the absence of an integrated, pre-deployment system that translates source code characteristics into complete, multi-dimensional cloud infrastructure configurations.

2.3 Domain Overview

Cloud computing, as defined by the National Institute of Standards and Technology (NIST), refers to a model for enabling ubiquitous, on-demand network access to a shared pool of configurable computing resources [13]. The evolution of this model — from Infrastructure as a Service (IaaS) through Platform as a Service (PaaS) to serverless computing — has progressively abstracted infrastructure management away from developers. Buyya et al. [14] identified the vision of cloud computing as a 'fifth utility', positioning it alongside electricity and water in terms of on-demand accessibility.

Within this domain, Infrastructure as Code (IaC) has emerged as a critical practice, enabling teams to define and manage infrastructure through machine-readable configuration files rather than manual processes. Terraform, developed by HashiCorp, is currently the most widely adopted IaC tool, offering a declarative syntax and support for over 300 cloud providers. Docker, the leading containerization platform, complements IaC by enabling applications to be packaged with their runtime dependencies into portable, lightweight container images.

The combination of cloud elasticity [20], IaC tooling, and containerization has created an environment of tremendous capability but equivalent complexity. Research in this domain is advancing rapidly, yet the specific challenge of pre-deployment, code-driven infrastructure recommendation remains largely unaddressed in the academic literature — a gap that this research directly targets.

2.4 Existing Systems, Frameworks, and Designs

The following comparative assessment examines the most relevant existing systems and frameworks. In each case, the suitability of the system to the problem investigated in this research is evaluated and justified.

2.4.1 Sharma et al. – Machine Learning-Based Resource Recommendation [4]

Sharma et al. proposed a machine learning approach to cloud resource recommendation using historical CPU and memory utilization data from live deployments. The system demonstrated meaningful accuracy in suggesting appropriate instance types for known workloads. However, the fundamental limitation of this approach is its dependency on existing runtime data — it cannot function for applications that have not yet been deployed. This renders it unsuitable for the pre-deployment use case that this research targets. The system also provides no IaC generation or cost estimation capabilities. Despite these shortcomings, the work provides a valuable reference for feature selection in ML-based recommendation models.

2.4.2 Chen et al. – Static Code Analysis for Memory Estimation [5]

Chen et al. developed a static analysis tool for Java applications that infers memory requirements by examining object allocation patterns in the codebase. This is the closest existing work to the code-driven analysis approach proposed in this research. The tool successfully demonstrated that infrastructure requirements can be predicted from source code without requiring runtime execution. However, its scope is narrow: it addresses only memory estimation, supports only the Java language ecosystem, and does not consider multi-provider deployments, security configurations, or cost optimization. This work validates the underlying approach of the proposed system but also highlights how much further the concept must be taken to deliver practical utility.

2.4.3 Zhang and Wang – CloudCostOpt [6]

CloudCostOpt is a post-deployment cost optimization platform that analyses cloud billing data and suggests resource rightsizing recommendations. While effective in its domain, it is explicitly reactive — it identifies inefficiencies only after they have already occurred and incurred cost. It provides no support for pre-deployment planning or IaC generation. This system is not suitable for the problem investigated in this research, as the goal is to prevent over- or under-provisioning before deployment, not to remediate it afterward.

2.4.4 Kim et al. – TerraGen [7]

TerraGen introduced template-based automatic generation of Terraform configurations for multi-cloud deployment. The system maps predefined application architecture patterns to corresponding Terraform templates, offering a meaningful step toward automating IaC authorship. However, its reliance on static, manually authored templates means it cannot adapt to the specific characteristics of individual applications. It produces the same output for any Node.js application regardless of its dependencies, database requirements, or expected scale. The proposed system in this research addresses this limitation by grounding IaC generation in ML-driven analysis of the specific repository.

2.4.5 Patel and Singh – Multi-Cloud Deployment Framework [9]

Patel and Singh proposed a framework for deploying applications across multiple cloud providers to mitigate vendor lock-in. Their work is relevant in establishing the multi-cloud context and demonstrating the technical feasibility of provider-agnostic deployment orchestration. However, the framework assumes infrastructure decisions have already been made and focuses on execution rather than recommendation. It is complementary to, rather than overlapping with, the scope of this research.

The comparative assessment above is summarized in the table below:

Study	Focus	Limitation Relative to This Research
Sharma et al. [4]	ML-based recommendation	Requires live runtime data; no IaC generation; single-dimension
Chen et al. [5]	Static code analysis	Java only; memory estimation only; no multi-cloud support
CloudCostOpt [6]	Post-deployment cost optimization	Reactive; no pre-deployment capability; no IaC support
TerraGen [7]	Template-based Terraform generation	Static templates; not application-specific; no ML component
Patel & Singh [9]	Multi-cloud orchestration framework	Assumes completed infrastructure decisions; no recommendation

2.5 Technological Analysis

2.5.1 Algorithmic Analysis

The machine learning approaches relevant to this research span classification and regression tasks. Multi-label classification is appropriate for instance type recommendation, where a repository may simultaneously require both a compute-optimized instance for an API server and a memory-optimized instance for a caching layer. Ensemble methods — particularly gradient boosting algorithms such as XGBoost — have been shown to outperform single models in structured tabular prediction tasks with heterogeneous feature spaces [17], making them well-suited for the feature-rich repository analysis

context. Transfer learning techniques, which have demonstrated success in domain adaptation tasks, are proposed to generalize models trained on one cloud provider's instance taxonomy to another, reducing the data required per provider.

For cost estimation, regression models are appropriate for predicting continuous quantities such as monthly compute costs. These models must account for complex pricing structures including on-demand, reserved, and spot pricing tiers, as well as regional price variations and supplementary service costs. The Total Cost of Ownership (TCO) estimation component will aggregate across these dimensions to produce comparable per-provider cost projections.

2.5.2 Design Analysis

The system architecture follows a microservices design pattern, with each functional module — repository analyser, recommendation engine, cost estimator, Terraform generator, Dockerfile generator, and web interface — implemented as a loosely coupled service. This design enables independent development, testing, and scaling of individual components. The repository analyser is designed around a pipeline pattern: each stage (dependency extraction, technology stack detection, architecture classification, feature normalization) transforms its input and passes the result to the next stage. This separation of concerns improves testability and allows individual pipeline stages to be replaced or upgraded without disrupting the overall workflow.

The IaC generation modules follow a template augmentation pattern rather than a purely template-based approach. A base Terraform template provides structural scaffolding, while ML-driven outputs populate variable fields including instance type, storage size, security group rules, and scaling policies. This hybrid design combines the reliability of validated templates with the adaptability of data-driven parameterization.

2.5.3 Workflow Analysis

The end-to-end workflow begins with a developer submitting a repository URL through the web interface. The repository analyser fetches the repository and executes the extraction pipeline, producing a structured feature vector. This vector is passed to the recommendation engine, which outputs ranked instance type recommendations per cloud provider. The cost estimator augments these recommendations with real-time pricing data. The IaC generators then produce Terraform and Dockerfile outputs based on the complete recommendation set. The web interface renders all outputs in a comparative, structured view and makes generated files available for download. This workflow is designed to complete within a target latency of under 60 seconds for typical repositories.

2.6 Reflection – Research Gap Justification

The literature reviewed above confirms that while meaningful progress has been made across the individual sub-domains of cloud resource recommendation, code analysis, IaC generation, and cost

optimization, no existing work integrates these dimensions into a unified, pre-deployment, code-driven recommendation system. All studies reviewed were published within the last six years, ensuring their relevance to the current state of cloud infrastructure practice.

The key gaps confirmed by this review are: the absence of pre-deployment recommendation capability; the lack of holistic multi-dimensional analysis combining performance, cost, and security; no intelligent cross-provider comparison grounded in application-specific code characteristics; no end-to-end system translating source code into complete IaC configurations; and no production-ready, fully implemented solution available for real-world developer use. This research addresses all five gaps through the design, development, and evaluation of the proposed automated cloud infrastructure recommendation system.

Chapter 03 – Methodology

3.1 Chapter Overview

This chapter presents the research methodology underpinning the study. It begins by establishing the philosophical position and research paradigm, followed by the chosen research approach and strategy. Data and fact collection mechanisms are then described, and the methodology execution workflow is laid out in a structured tabular format. The chapter also addresses project management methodology selection, the project timeline with key milestones, ethical considerations, and a chapter summary.

3.2 Research Paradigm

This research is positioned within the pragmatist paradigm. Pragmatism is selected because the primary goal of the study is not to test a universal theory but to solve a specific, real-world problem — the difficulty developers face in selecting appropriate cloud infrastructure before deployment. The pragmatist stance is problem-centred and outcome-oriented, judging the value of research methods by their ability to produce useful, actionable results rather than by adherence to a fixed ontological or epistemological position [12].

From an ontological perspective, this research accepts that cloud infrastructure requirements are partially objective (resource metrics, pricing data) and partially subjective (expert judgement on 'optimal' configurations). Epistemologically, knowledge about the relationship between application code and infrastructure requirements is best generated by combining empirical data analysis (quantitative) with expert insight (qualitative) — a mixed-methods stance consistent with pragmatism. This dual perspective allows the research to benefit from the statistical rigour of machine learning model evaluation while also incorporating the contextual nuance that expert interviews and usability studies provide.

3.3 Research Approach

This research adopts a mixed-methods approach combining inductive and deductive reasoning. The deductive component draws from existing literature on cloud resource management, code analysis, and machine learning to derive hypotheses about which repository features are most predictive of infrastructure requirements. These hypotheses are then operationalized into a machine learning model trained on empirical data.

The inductive component emerges from the evaluation phase, where patterns observed in the system's recommendation outputs — compared against expert decisions and real-world deployment outcomes — generate new insights about the relationship between application code characteristics and infrastructure needs. This iterative interplay between deduction and induction is consistent with the Design Science

Research paradigm and supports the progressive refinement of the recommendation engine over the course of the project.

3.4 Research Strategy

The primary research strategy is Design Science Research (DSR), as defined by Peffers et al. [12]. DSR is particularly appropriate for this study because it is explicitly oriented toward the creation of innovative IT artifacts — in this case, a functional software system — as a means of solving identified practical problems. DSR's iterative development-and-evaluation cycles align naturally with the incremental build approach planned for this project.

Complementary strategies include Experimental Research, employed during the system evaluation phase to compare recommendation outputs against expert baselines using controlled datasets, and Case Study Research, used to examine how the system performs across representative real-world repositories spanning different technology stacks and project scales.

3.5 Fact Collection Mechanisms

Data will be gathered from five sources:

- GitHub Repository Dataset: Approximately 1,000 public repositories across common web application frameworks, collected via the GitHub API. Repositories will be filtered to include only those with open-source licenses permitting research use and those with documented deployment configurations.
- Cloud Pricing Data: Real-time pricing data obtained from the official APIs of AWS, Azure, GCP, and DigitalOcean, covering compute instances, managed storage, network egress, and relevant supplementary services.
- Expert Decision Dataset: Structured interviews and questionnaires administered to 20 experienced cloud infrastructure professionals, each providing infrastructure recommendations for 50 sample applications. This dataset serves as the evaluation ground truth.
- Performance Benchmark Data: Published benchmark data (including SPEC CPU results) for representative cloud instance types, used to validate the performance-appropriateness of recommended configurations.
- User Feedback: Usability testing sessions with 30 developers spanning varying levels of cloud experience, producing System Usability Scale (SUS) scores and qualitative feedback on the interface and recommendation quality.

3.6 Research Methodology Execution Workflow

The research follows the six-stage DSRM process. The table below maps each stage to its activities, outputs, and the research objectives it addresses:

DSRM Stage	Activities	Deliverables
1. Problem Identification	Literature review; developer interviews; industry surveys documenting cloud infrastructure challenges	Annotated bibliography; problem definition document; research questions
2. Relevance Justification	Gap analysis against reviewed literature; validation of problem scope through expert consultation	Confirmed research gap statement; refined objectives
3. Comparative Analysis and Gap Justification	Systematic comparison of existing systems (Sharma [4], Chen [5], Kim [7], etc.) across evaluation dimensions	Comparative assessment table; justified research gap
4. Define and Finalize Objectives	Derivation of SMART research objectives from problem definition; stakeholder requirement gathering	Finalized objectives (Obj. 1–4); requirements specification
5. Design, Development, and Data Management	System architecture design; iterative development of all six modules; dataset collection and preprocessing; ML model training	MVP system; trained ML models; Terraform and Dockerfile generators; integrated web interface
6. Evaluation and Communication	Expert comparison study; usability testing; performance benchmarking; dissertation write-up and dissemination	Evaluation report; final dissertation; potential academic publication

3.7 Project Management Methodology

This project will be managed using the Scrum framework, selected for its alignment with the iterative, artifact-centred nature of Design Science Research. Scrum's sprint structure — with each sprint producing a potentially shippable increment — maps naturally onto the progressive development of system modules: the repository analyser, recommendation engine, cost calculator, IaC generators, and web interface can each be developed, tested, and demonstrated in successive sprints without requiring the complete system to be built before any component is evaluated.

An alternative considered was the Waterfall model, which would have provided a cleaner separation between requirements, design, and implementation phases. However, Waterfall's linear progression is poorly suited to research-driven development, where early evaluation findings often necessitate revisions to design decisions made in earlier phases. Scrum's built-in retrospective and backlog refinement cycles accommodate this kind of adaptive evolution, making it the more appropriate choice for this project. Sprint duration will be set to two weeks, with a product backlog maintained in GitHub Projects and sprint reviews used to validate incremental outputs against research objectives.

3.8 Project Timeline

Phase	Activities	Duration	Deliverables

Phase 1: Foundation	Literature review, Problem definition, Requirements gathering	Nov 2025 - Dec 2025	Research proposal, Annotated bibliography, Requirements specification
Phase 2: Design	System architecture design, Data model design, UI/UX design	Jan 2026 - Feb 2026	Architecture diagrams, Database schema, UI prototypes
Phase 3: Development I	Repository analyzer, Basic recommendation engine, Web interface	Mar 2026 - Apr 2026	MVP system, Initial ML models, Functional UI
Phase 4: Development II	Cost calculator, Terraform generator, Dockerfile generator	May 2026 - Jun 2026	Complete core functionality, Integration testing
Phase 5: Evaluation	Expert comparison study, User testing, Performance evaluation	Jul 2026 - Aug 2026	Evaluation report, User feedback analysis, System metrics
Phase 6: Refinement	Model optimization, UI improvements, Documentation	Sep 2026 - Oct 2026	Final system, Complete documentation, Research paper
Phase 7: Finalization	Thesis writing, Final presentation preparation	Oct 2026 - Nov 2026	Final dissertation, Presentation materials

3.9 Ethical Considerations

This research has been designed in full compliance with established ethical standards across the following dimensions:

- Data Privacy: GitHub access tokens and user session data will be encrypted in transit and at rest. Private repository code will be processed in memory only and never persisted. All participants' data from interviews and usability studies will be anonymized before analysis.
- Intellectual Property: Only open-source repositories carrying licenses that explicitly permit research use will be included in the training dataset. Generated IaC outputs will include appropriate licensing notices. No proprietary code will appear in research publications.
- Informed Consent: All participants in expert interviews and usability testing sessions will provide written informed consent prior to participation. They will be clearly informed of how their data will be used and will retain the right to withdraw at any point without consequence.
- Algorithmic Fairness: The training dataset will be curated to include diverse technology stacks and application types, minimizing the risk of systematic bias toward specific frameworks or languages. Recommendations will be audited periodically for bias patterns during the development cycle.

- Environmental Responsibility: Carbon efficiency will be incorporated as a criterion in cross-provider cost comparisons, providing developers with visibility into the environmental impact of their infrastructure choices in addition to financial cost.
- Academic Integrity: All referenced works will be properly cited. System capabilities and evaluation results will be reported accurately and without exaggeration.

3.10 Chapter Summary

This chapter established the methodological foundation of the research. The pragmatist paradigm was selected and justified as the appropriate philosophical stance for a problem-centred, artifact-driven study. A mixed inductive-deductive approach underpins the research design, and Design Science Research was identified as the primary strategy, complemented by experimental and case study elements. Five data collection mechanisms were described, and the DSRM execution workflow was mapped across six structured stages linking activities to deliverables and objectives. Scrum was selected as the project management methodology based on its compatibility with iterative, research-driven development. Ethical considerations were addressed across privacy, consent, fairness, and integrity dimensions. The subsequent chapter will present the full system requirements specification, including stakeholder analysis, UML diagrams, and functional and non-functional requirements.

References

[1] M. Armbrust et al., "A view of cloud computing," Communications of the ACM, vol. 53, no. 4, pp. 50–58, Apr. 2010.

[2] A. Li, X. Yang, S. Kandula, and M. Zhang, "CloudCmp: comparing public cloud providers," in Proc. 10th ACM SIGCOMM Conf. Internet Meas., 2010, pp. 1–14.

[3] Cloud Native Computing Foundation, "CNCF Annual Survey 2024," [Online]. Available: https://www.cncf.io/reports/cncf-annual-survey-2024/

[4] P. Sharma, L. Guo, X. He, and D. Irwin, "Flint: batch-interactive data-intensive processing on transient servers," in Proc. 11th Eur. Conf. Comput. Syst., 2016, pp. 1–15.

[5] T. Chen, X. Zhang, and S. Liu, "Performance prediction for cloud-based applications using machine learning," IEEE Trans. Cloud Comput., vol. 8, no. 4, pp. 1223–1235, Oct. 2020.

[6] Q. Zhang and R. Wang, "CloudCostOpt: a cost optimization tool for cloud computing environments," in Proc. IEEE 7th Int. Conf. Cloud Comput. Technol. Sci., 2015, pp. 278–283.

[7] J. Kim, S. Lee, and H. Kim, "TerraGen: automatic generation of Terraform configurations for multi-cloud deployment," in Proc. IEEE 13th Int. Conf. Cloud Comput., 2020, pp. 412–419.

[8] A. Gupta, L. Kalé, D. Milojicic, and P. Faraboschi, "HPC cloud for scientific and business applications: taxonomy, vision, and research challenges," ACM Comput. Surv., vol. 51, no. 1, pp. 1–29, Jan. 2018.

[9] R. Patel and A. Singh, "A framework for multi-cloud deployment and management of web applications," J. Cloud Comput., vol. 9, no. 1, pp. 1–18, Dec. 2020.

[10] W. Li, Y. Chen, and H. Wang, "Optimizing Docker containers for cloud-native applications," in Proc. IEEE Int. Conf. Cloud Eng., 2019, pp. 229–234.

[11] A. Rahman, C. Parnin, and L. Williams, "The seven sins: security smells in infrastructure as code scripts," in Proc. IEEE/ACM 41st Int. Conf. Softw. Eng., 2019, pp. 164–175.

[12] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," J. Manage. Inf. Syst., vol. 24, no. 3, pp. 45–77, Dec. 2007.

[13] L. Youseff, M. Butrico, and D. Da Silva, "Toward a unified ontology of cloud computing," in Proc. Grid Comput. Environ. Workshop, 2008, pp. 1–10.

[14] R. Buyya, C. S. Yeo, S. Venugopal, J. Broberg, and I. Brandic, "Cloud computing and emerging IT platforms: vision, hype, and reality for delivering computing as the 5th utility," Future Gener. Comput. Syst., vol. 25, no. 6, pp. 599–616, Jun. 2009.

[17] D. Tran, H. Nguyen, and M. Zhao, "Machine learning-based resource allocation for cloud data centers," IEEE Trans. Cloud Comput., vol. 9, no. 2, pp. 567–580, Apr. 2021.

[20] N. R. Herbst, S. Kounev, and R. Reussner, "Elasticity in cloud computing: what it is, and what it is not," in Proc. 10th Int. Conf. Auton. Comput., 2013, pp. 23–27.